Axral Codex Web版 ユーザーガイド

Worldbuilding Compiler & World State Engine ― ブラウザ版（React / Vite）操作マニュアル

対象: src/ 以下のWeb UI（PyQtデスクトップ版とは別実装） | 準拠仕様: plan.md v2

目次

1. Axral Codexとは
2. 画面構成の全体像
3. Codex Syntax（本文の書き方）
4. グラフキャンバス（中央パネル）の操作
5. Candidate Suggestions（右パネル）
6. Header メニュー（File / Patches / Templates）
7. Non-destructive Text Sync と Conflict解決
8. Timeline Lens（下部パネル）
9. Worldbuilding Linter（下部パネル）
10. データの保存・読み込み・書き出し
11. 現時点での既知の制約（スコープ外の機能）
12. 困ったときは（トラブルシューティング）

1. Axral Codexとは

Axral Codexは、物語・世界観設定（ワールドビルディング）のための「テキスト → グラフ」コンパイラです。 左側のエディタに専用の軽量記法（Codex Syntax）でキャラクターや組織などの登場物（Entity）とその関係（Relation）を書くと、 中央のグラフキャンバスにリアルタイムで世界の相関図が描画されます。

AI/NLPの解釈に依存せず、明示的に書かれたCodex Syntaxのみを確実にデータへ変換する「決定論的パーサー」が中核にあります。 一方で、地の文（自然文）から関係の候補を推測して提案する「Secondary Indexer」、 時間経過にともなう世界の変化を扱う「Timeline Lens」、矛盾を検出する「Worldbuilding Linter」も備えています。

> 本ガイドはブラウザで動作するWeb版（React / Vite製、 `src/` 以下）を対象としています。 同リポジトリにはPyQtによるデスクトップ版（ `main.py` / `ui/` 以下）も同梱されていますが、操作方法が一部異なるため対象外です。

2. 画面構成の全体像

画面は上から順に、以下の5つのエリアで構成されています。

エリア	内容
① ヘッダーメニューバー	File / Patches / Templates メニューと「Reset Sample」ボタン
② 左パネル：Codex Syntax エディタ	世界観データを記述するテキストエディタ（幅 約30%）
③ 中央パネル：Graph Canvas	Entity/Relationを描画するインタラクティブな相関図（可変幅）
④ 右パネル：Candidate Suggestions	地の文から抽出された関係候補の提案リスト（幅 約28%）
⑤ 下部パネル：Timeline Lens / Worldbuilding Linter	タブ切り替え式。時系列操作とデータ整合性チェック
⑥ ステータスバー	最下部。Entity数・Relation数・候補数・矛盾件数などのサマリ

編集内容は自動的にブラウザの `localStorage` へ保存されるため、ページを再読み込みしても直前の編集内容が復元されます。

3. Codex Syntax（本文の書き方）

左側のエディタに以下のような記法でテキストを書くと、中央のグラフが自動更新されます（入力から約150msのデバウンス後に再解析）。

```
[Alice:Character:Su-ken]
[Bob:Character:Su-ken]
[Order:Organization:Kan-ken]

[Alice:Character] -> [Order:Organization] : 所属
[Bob:Character] -> [Order:Organization] : 所属
Alice -> Bob : 友好

Bobはかつてaliceを尊敬していた。しかしOrderの教義を巡って対立した。
```

3.1 構文一覧

記法	意味
`[名前:型]`	Entity宣言（例: `[Alice:Character]`）
`[名前:型:Layer]`	Layer付きEntity宣言。Layer Sliderでの表示/非表示に使う
`[A:型] -> [B:型] : 関係名`	型を明示した関係（Relation）
`A -> B : 関係名`	型を省略したshorthand。既出のEntity名を参照する。型は自動的に "Unknown" になり、後から `[A:型]` の宣言があれば確定する
それ以外の行（地の文）	自然文として扱われ、Secondary Indexerが関係候補を抽出する
`# Generated Relations`	アプリが自動生成する行を追記する専用セクション見出し（手で書く必要はない）
`PATCH Source->Target#n: type = "値" # was: "旧値"`	グラフキャンバス上での編集から自動生成されるPatchブロック

3.2 Entityの色分け（Layer/型）

型	色
Character	青（#5B8DEF）
Organization	橙（#E0A73C）
Location	緑（#5FB878）
Unknown（型未確定）	灰（#9AA0A6）

3.3 Relationの線種（確からしさ）

status	線種
confirmed（既定）	実線
hypothetical	破線
rumor	点線（薄いグレー）

statusはTimeline Lensの「Add Event...」から `relation_status` Effectを追加することで変化させられます（詳細は第8章）。

4. グラフキャンバス（中央パネル）の操作

4.1 視点操作

操作	方法
パン（視点移動）	背景を左クリックしたままドラッグ / 右クリックまたは中クリックのドラッグ / 2本指スクロール
ズーム	`Ctrl` + ホイール、またはトラックパッドのピンチ操作
ノード移動	ノード（円）を左クリックしたままドラッグ
キーボード移動	`↑` `↓` `←` `→` または `W` `A` `S` `D`（`Shift` で大きく移動）

全体表示にフィット	ツールバーの拡大アイコン、画面右下D-Padの「FIT」ボタン、または `Ctrl`+`0`
カメラを原点(0,0)にリセット	ツールバーの照準アイコン
円形レイアウトに再配置	ツールバーの再読み込みアイコン

タッチ操作にも対応しており、1本指でノードドラッグ／パン、2本指でピンチズーム＋パンが行えます。

4.2 Relationタイプの編集（Canvas → Patch）

グラフ上のRelationラベル（丸みのあるバッジ）をクリックまたはダブルクリックすると、「Relationを編集」ダイアログが開き、新しい関係タイプを入力できます。

> ここで確定しても、本文（Original Text）はその場では書き換わりません。エディタ末尾の `# Generated Relations` セクションに `PATCH` ブロックとして追記されるだけです（Non-destructive Text Sync、第7章参照）。

4.3 画面右上の凡例・情報

右上に Character（青）／Organization（橙）／Location（緑）の凡例と、現在表示中のノード数・エッジ数が表示されます。

5. Candidate Suggestions（右パネル）

エディタの地の文（自然文）に既知のEntity名が2つ登場し、かつ関係を示すキーワードが含まれる文があると、 Secondary Indexerがそれを解析して「関係候補（Candidate）」として右パネルに表示します。

5.1 候補カードの見方

- **見出し行**: `Source ── 関係タイプ ──▶ Target` の形式で表示
- **confidenceバッジ**: HIGH（緑）／MEDIUM（黄）／LOW（赤）の3段階と、0〜1のスコア値
- **line番号**: 候補の元になった行番号
- **⚠ ambiguous バッジ**: 1文に3つ以上のEntityが登場するなど曖昧な場合に表示
- **引用文**: 元になった地の文がそのまま表示される

5.2 Accept / Ignore

ボタン	動作
Accept	本文末尾の `# Generated Relations` セクションへ確定したCodex Syntax行（例: `A -> B : 所属`）を追記し、パーサーに再解釈させる。Canonical Dataを直接書き換えることはない
Ignore	その候補をセッション中は非表示にする。本文にもCanonical Dataにも一切影響しない

> Secondary Indexerは形態素解析を行わない素朴な部分文字列一致方式です。代名詞（彼・それ等）で先行詞が特定できない場合や、 3つ以上のEntityが1文に登場する曖昧な文は、候補を生成せずLinterパネルの「Ambiguity」として通知されます。

6. Header メニュー（File / Patches / Templates）

6.1 File メニュー

項目	内容
Open Codex Text...	ローカルの `.txt` / `.md` / `.codex` ファイルを読み込み、エディタの内容を置き換える
Save Codex Text (.txt / .md)	現在のエディタ内容を `world_codex.md` としてダウンロード
Export Canonical Data (JSON)...	解析済みのEntity/RelationデータをJSON（`world_canonical.json`）として書き出す

6.2 Patches メニュー

「Resolve Conflicts...」を選ぶと、本文とPatchの間で矛盾が発生している箇所を解消するダイアログが開きます（第7章参照）。 矛盾が1件以上ある間は、メニューボタンがオレンジ色でハイライトされ、件数バッジが表示されます。

6.3 Templates メニュー

よく使う構文をカーソル位置ではなく本文末尾に挿入できます。

- `[名前:型:Layer]`（Entity宣言）
- `[A:型] -> [B:型] : 関係`（明示Relation）
- `A -> B : 関係`（Shorthand）

6.4 Reset Sample

右上の「Reset Sample」ボタンで、エディタ内容・Event一覧・Ignore済み候補をすべて初期サンプルに戻せます（確認ダイアログあり、元に戻せません）。

7. Non-destructive Text Sync と Conflict解決

Axral Codexの重要な原則は「人が書いたテキストを絶対に破壊しない」ことです。グラフキャンバス上での編集は、 本文を直接書き換えるのではなく、末尾の `# Generated Relations` セクションに `PATCH` ブロックとして追記されます。

```
PATCH Alice->Order#0:
  type = "対立"  # was: "所属"
```

再解析のたびに、この `was` 値と本文（Original Text）側の現在の値が比較されます。一致していればPatchはそのまま適用され、 グラフに反映されます。もし本文側の値がPatch作成後に変わっていた場合は**Conflict（競合）**として検出され、Patchは適用されません。

7.1 Conflict Resolutionダイアログの選択肢

Patchesメニューの「Resolve Conflicts...」から開くダイアログでは、各Conflictについて以下の4つから選べます。

選択肢	動作
Keep Text	Patchブロックを削除し、本文（Original Text）の値をそのまま正とする
Apply Canvas	本文の該当行をPatchの値で直接上書きし、Patchブロックを削除する
Merge	OriginalともPatchとも異なる第三の値を自分で入力し、本文に直接書き込む。Patchブロックは削除される
Skip	今は解決せず、ダイアログを閉じる（後で再度開いて解決できる）

ダイアログ上部には対象のRelation（ `ref_key` ）とフィールド名、対応する行番号が表示され、 本文側の値（Text）とPatch側の値（Canvas編集値）、Patch作成時点の想定値（ `expected_was` ）が並べて確認できます。 複数のConflictがある場合はページ送りで1件ずつ処理します。

8. Timeline Lens（下部パネル）

下部パネルの「Timeline Lens」タブでは、Event（出来事）を積み重ねて世界の時系列変化を確認できます。

8.1 Timeline State を表示する

チェックボックスをONにすると、グラフキャンバスの表示が「現在の本文（Live）」から、選択した時点までのEventを適用した 「World State」（ `WorldStateEngine` の計算結果）に切り替わります。

8.2 スクラバー（タイムスタンプスライダー）

-50 〜 +50 の範囲でタイムスタンプを移動できます。現在値は `t = 数値` の形式でリアルタイム表示されます。

8.3 Change Summary（Before → After）

選択中のタイムスタンプに発生するEventがあれば、その直前（t-1）と直後（t）のWorld Stateを比較し、 「〇〇.existence: "confirmed" → "dead"」のような差分がテキストで一覧表示されます。

8.4 Layer Slider

Entity宣言時の `[名前:型:Layer]` で指定したLayerごとに、グラフ上の表示/非表示をチェックボックスで切り替えられます。 Layerを指定していないEntityは常に表示されます。

8.5 Eventの追加・削除

「Add Event...」ボタンから、以下の項目を指定してEventを追加します。

項目	内容
Timestamp	Eventが発生する時点（整数）
Type	自由入力（例: Tear, Chapter, MajorEvent）
Snapshot Point にする	チェックすると、その時点のWorld Stateをキャッシュ対象にする
Effect kind	`entity_existence` （Entityの存在・生死）または `relation_status` （Relationの確からしさ）
対象 (Target)	本文から解析済みのEntity名、またはRelationの `ref_key` をプルダウンから選択

新しい値 (New Value)	entity_existenceなら例: `dead` / `confirmed`、relation_statusなら `confirmed` / `hypothetical` / `rumor`

追加したEventはイベント一覧に「t=数値 [Type]」の形式で表示され、選択して「Delete Selected」で削除できます。

> Timelineで追加したEventは、本文（Codex Syntax）やCanonical Dataには一切保存されません。ブラウザのセッション内メモリにのみ存在するため、 ページを再読み込みすると失われます（永続化は今後の拡張予定）。

9. Worldbuilding Linter（下部パネル）

下部パネルの「Worldbuilding Linter」タブには、再解析のたびに自動実行された検証結果の一覧が表示されます。 項目をクリックすると、対応する行がある場合はエディタの該当行にジャンプします。

9.1 検出される問題の種類

- **Parser issue** — 未知のLayer指定、Patch対象が見つからない、など
- **Patch Conflict** — 本文とPatchの値の食い違い（第7章）
- **Ambiguity** — Secondary Indexerが解決できなかった参照や、複数Entityが登場する曖昧な文
- **Temporal Contradiction** — Entityが死亡等の状態になった後の時点で、別のEventに参加している
- **Relation Date Contradiction** — Relationの `valid_from` / `valid_to` が数値として矛盾している（ `valid_to < valid_from` ）
- **Missing Reference** — Effectが、その時点のWorld Stateに存在しないEntityを参照している
- **Duplicate Entity** — 同じ名前のEntityが異なる型で複数回宣言されている

件数バッジは警告（amber）があればオレンジ色、情報（info）のみなら青色で表示されます。問題が0件のときは 「問題は見つかりませんでした✓」と表示されます。

10. データの保存・読み込み・書き出し

操作	方法
自動保存	編集内容は入力のたびにブラウザの `localStorage` に自動保存される（キー: `axral_codex_text` ）
テキストとして保存	File > Save Codex Text（ `world_codex.md` としてダウンロード）
テキストを読み込む	File > Open Codex Text...（ `.txt` / `.md` / `.codex` ファイルを選択）
構造化データの書き出し	File > Export Canonical Data (JSON)...（ `world_canonical.json` ）

> JSONエクスポートはEntity/Relationの構造化データのみが対象です。Event（Timeline Lensで追加したもの）は含まれません。

11. 現時点での既知の制約（スコープ外の機能）

> 以下はplan.mdで定義されている機能のうち、現バージョンでは意図的に未実装のものです。
>
> - Confidence算出へのEmbedding Signal / LLM Signalの追加（現在はルールベースのみ）
> - Infinite Canvas最適化・Undo/Redo
> - ノードを2枚並べるフル版のBefore/After View（現状はテキストの変化サマリのみ）
> - Membership Contradiction検出（参加/離脱専用のEffectが未実装のため）
> - 一般的なRelation Contradiction検出（対義語辞書などのドメインオントロジーが必要なため、現状は日付の数値矛盾のみ検出）
> - Timelineで追加したEventの永続化（本文への書き込み）
> - 複数ユーザーによるリアルタイム共同編集（CRDT）
> - Incremental Parsing・Viewport Culling等の大規模データ向け性能最適化（現状は毎回全文を再解析）

12. 困ったときは（トラブルシューティング）

症状	対処

グラフに何も表示されない	エディタに `[名前:型]` 形式のEntity宣言が1つも無いか確認する。空の状態では「Codex Syntaxを入力するとここに世界が図になります」と表示される
関係候補（Candidate）が出てこない	地の文に、既に宣言済みのEntity名が正確な文字列で2つ含まれているか、関係を示すキーワードが含まれているか確認する
Relationの編集が反映されない	Patches > Resolve Conflicts... を開き、Conflictが発生していないか確認する。本文側が後から変更されているとPatchは自動適用されない
ノードが探しているのに見つからない	Layer Sliderで該当Layerが非表示になっていないか確認する。またはツールバーの「全体表示」（拡大アイコン）でフィットさせる
編集内容が消えた	別ブラウザ・別デバイス・シークレットウィンドウでは `localStorage` が共有されないため、消えたように見える。File > Save Codex Textでこまめにバックアップを取ることを推奨
Timelineで追加したEventが消えた	Eventはセッション内メモリのみに保持され、永続化されない仕様（第8.5節参照）。ページ再読み込みで失われる